

Large file upload

Insert as section 9 of the solution document, before Known limitations. Renumber the sections that follow, and apply the three amendments listed at the end.

9. Large file upload

9.1 The problem

The automatic sync described in section 2 moves a file through Apex: the code reads the file out of Salesforce, then packages it for SharePoint. Apex has a fixed 12 MB working memory allowance for background processing, and the file occupies that allowance twice — once as it is read and once as it is packaged — after being converted to a text encoding that adds roughly a third to its size.

A 6 MB document therefore needs around 16 MB of a 12 MB allowance. Files above approximately 6 MB cannot be moved this way.

This is a Salesforce platform limit that applies to every customer. It is not affected by how the code is written, and no configuration setting increases it.

Salesforce itself accepts attachments up to 2 GB, so a user can attach a document that the sync cannot move. The **Max File Size Bytes** setting exists so that those files are identified and recorded rather than failing with an unhelpful platform error.

9.2 The approach

The file is sent from the user's browser straight to SharePoint, without passing through Apex at all. Apex is responsible only for the small requests at either end.

[Screenshot 15 — Sequence diagram] *Insert a diagram with three lanes: Browser, Apex, SharePoint. Show Apex requesting an upload session, the URL being returned to the browser, the file travelling directly from browser to SharePoint, then the browser reporting the file id back to Apex. Emphasise that the file itself never crosses the Apex lane.*

The sequence:

1. The user selects a file using the **Upload to SharePoint** button on the Opportunity record page.

2. Apex asks SharePoint to open an upload session for that filename in that record's folder. This is a small JSON request; the file is not involved.
3. SharePoint returns a one-time upload address.
4. Apex passes that address to the browser.
5. The browser sends the file directly to SharePoint in 5 MB pieces. Each piece carries a byte range so SharePoint can reassemble the document.
6. The final piece returns the created document.
7. The browser tells Apex the document's identifier, and Apex completes the same follow-up work the automatic sync performs: writing the library columns, granting the record owner access, and recording the folder link on the Opportunity.

Because the file never occupies Apex memory, the 6 MB limit does not apply. Files of 50 MB and above upload successfully.

9.3 The upload address

The address returned in step 3 is not a general purpose credential. It has four properties worth recording:

Property	Detail
Pre-authenticated	Authorisation is embedded in the address. The browser attaches no token of its own, and the application's own credentials never leave the Salesforce server
Single destination	Bound to one document library, one folder, and one filename. It cannot be redirected to another location
Write only	Grants no ability to read any document, including the one being written
Short lived	Expires approximately 15 minutes after being issued

What reaches the browser is therefore a time-limited permission to write one specific file to one specific place, not access to the SharePoint site.

9.4 Transfer mechanism

Each piece is sent with a byte range identifying its position in the file. SharePoint acknowledges each piece and states which range it expects next, returning the completed document when the final piece arrives.

Two consequences follow from this design:

- **Progress is measurable.** The component displays a progress bar because it knows how many bytes have been acknowledged.

- **The transfer is resumable.** A dropped connection does not require the upload to restart from the beginning, because the session records which ranges have been received.

Piece size is 5 MB, which satisfies Microsoft's requirement that every piece except the last be a multiple of 320 KiB.

9.5 The Salesforce copy

A file uploaded this way is stored in SharePoint. It is not stored in Salesforce, for the same reason that gave rise to the feature: a file too large for Apex to read is also too large for Apex to write.

To keep the user experience consistent, the integration adds a **link entry** to the record's Files related list. Salesforce supports a link type file, which behaves like any other file in the related list but points at an external address rather than holding content. It has no size limit.

[Screenshot 16 — Files related list showing the link entry] *Show the uploaded document appearing in the Opportunity's Files list, and note that opening it takes the user to SharePoint.*

The practical differences between a link entry and a stored file:

	Stored file (automatic sync)	Link entry (large upload)
Size limit	Approximately 6 MB	None
Held in Salesforce	Yes	No
Preview in Salesforce	Yes	No — opens in SharePoint
Requires SharePoint access to open	No	Yes
Survives deletion in SharePoint	Yes	No — becomes a broken link

This should be explained to users, because the two upload routes behave differently. See the recommendation in section 9.8.

9.6 Components

Type	Name	Purpose
Apex class	<code>SharePointLargeFileController</code>	Opens the upload session and performs the follow-up work

LWC	<code>sharepointFileViewer</code>	Upload button, progress bar, and the piece-by-piece transfer
Custom metadata field	<code>Create_Salesforce_Link__c</code>	Controls whether a link entry is added to Salesforce Files

`SharePointLargeFileController` is a new class. No existing class was modified to add this feature; the new class reuses the existing public methods for folder naming, column writing, and permissions.

9.7 Configuration

One additional platform setting is required, because browsers block requests from a Lightning page to an external address unless that address is explicitly permitted.

[Screenshot 17 — CSP Trusted Site] *Setup → CSP Trusted Sites. Show the entry with connect-src enabled.*

Field	Value
Trusted Site Name	<code>SharePoint_Upload</code>
URL	<code>https://bankunited.sharepoint.com</code>
Active	Yes
Context	All
CSP Directive	connect-src

Without this entry the browser silently refuses the transfer, and the failure appears in the browser console rather than as a Salesforce error.

One additional configuration field:

Field	Value	Effect
Create Salesforce Link	Checked	Adds a link entry to the record's Files list after upload

No change to the Entra ID application permissions is required. The upload session is created with the same application permission already granted for the automatic sync.

9.8 Recommendation on user experience

The record page now offers two ways to add a document, with different behaviour:

1. The standard **Files** related list — the document is stored in Salesforce and copied to SharePoint automatically, up to approximately 6 MB.
2. The **Upload to SharePoint** button — any size, stored in SharePoint with a link entry in Salesforce.

Presenting two routes without explanation will cause confusion. Two options:

- **Label the distinction.** Rename the button to make the behaviour self-evident, for example "Upload large file to SharePoint". Lowest effort, and sufficient if the large file route is genuinely occasional.
- **Consolidate on one route.** Direct users to the component for all uploads, accepting that documents then live in SharePoint with a link in Salesforce regardless of size. Simpler to explain, at the cost of losing in-Salesforce preview for small documents.

This is a business decision rather than a technical one and should be confirmed with the Wholesale Banking team before wider rollout.

9.9 Testing

Area covered	Assertion
Upload session created	Session requested for the correct folder and filename
Existing document protected	Session requests a non-destructive conflict behaviour
Missing folder	Folder chain created and the session request retried
Follow-up work	Columns written with the file name and uploader email
Column failure	Reported as a warning; the document remains in SharePoint
Link entry	Created as a link type with no content stored in Salesforce
Loop prevention	The link entry does not trigger the automatic sync
Validation	Missing record id or document id is rejected

[Screenshot 18 — A large file uploaded successfully] *Show the progress bar during upload, or the success message, together with the file visible in SharePoint. State the file size used in the test.*

Amendments to existing sections

Section 1, Scope. Move "Files larger than the configured size limit" out of the out of scope list and add to the in scope list: "Upload of files above the Apex size limit directly from the browser to SharePoint".

Section 9 Known limitations (now section 10), item 1. Amend the recommendation. The 6 MB ceiling still applies to the automatic sync for files attached through the standard Files related list, but a route now exists for larger documents. The remaining decision is whether the two-route experience is acceptable or whether the automatic sync should also be moved to middleware.

Section 10 Known limitations, new item. Add: "Documents uploaded through the large file route are stored in SharePoint only. Salesforce holds a link entry rather than a copy, so opening the document requires SharePoint access and the entry becomes invalid if the document is deleted in SharePoint."

Appendix B, Component inventory. Add the three rows listed in section 9.6.

Appendix addition — upload session API calls

Add to Appendix A as sections A.6 and A.7.

A.6 Open an upload session

Request

None

```
POST /drives/<DRIVE-ID>/root:/Opportunities/{Opportunity Name} -  
{Record Id}/{filename}:/createUploadSession  
Content-Type: application/json
```

```
{  
  "item": {  
    "@microsoft.graph.conflictBehavior": "rename"  
  }  
}
```

Response

JSON

200 OK

```
{
  "uploadUrl":
  "https://bankunited.sharepoint.com/_api/v2.0/drives/<DRIVE-ID>/items/<ITEM-ID>/uploadSession?guid=<GUID>&overwrite=False&rename=True&access_token=<TOKEN>",
  "expirationDateTime": "2026-09-25T10:15:00Z"
}
```

The `uploadUrl` embeds its own authorisation and must be treated as a credential: it is passed to the browser for the duration of the transfer and must not be logged or stored.

A.7 Send a piece of the file

Sent by the browser, not by Apex. No `Authorization` header is attached; SharePoint rejects the request if one is present.

Request

None

```
PUT {uploadUrl}
Content-Range: bytes 0-5242879/47185920

<5 MB of file content>
```

Response, intermediate pieces

JSON

202 Accepted

```
{
  "expirationDateTime": "2026-09-25T10:15:00Z",
  "nextExpectedRanges": [ "5242880-" ]
}
```

Response, final piece

JSON

201 Created

```
{
  "id": "<ITEM-ID>",
  "name": "big-scan.pdf",
  "size": 47185920,
  "webUrl":
  "https://bankunited.sharepoint.com/sites/WB/WB%20Salesforce%20%20PO
  C/Opportunities/...",
  "file": { "mimeType": "application/pdf" },
  "createdBy": {
    "user": { "displayName": "SharePoint App" },
    "application": { "id": "<CLIENT-ID>", "displayName":
    "<APP-NAME>" }
  }
}
```

`nextExpectedRanges` is what makes the transfer resumable: an interrupted upload can query the session for the ranges still outstanding rather than starting again.